One-Legged Hopper

Hybrid Dynamics Trajectory Optimization

Class Project: 01208583 Robotics | Semester 2, 2025

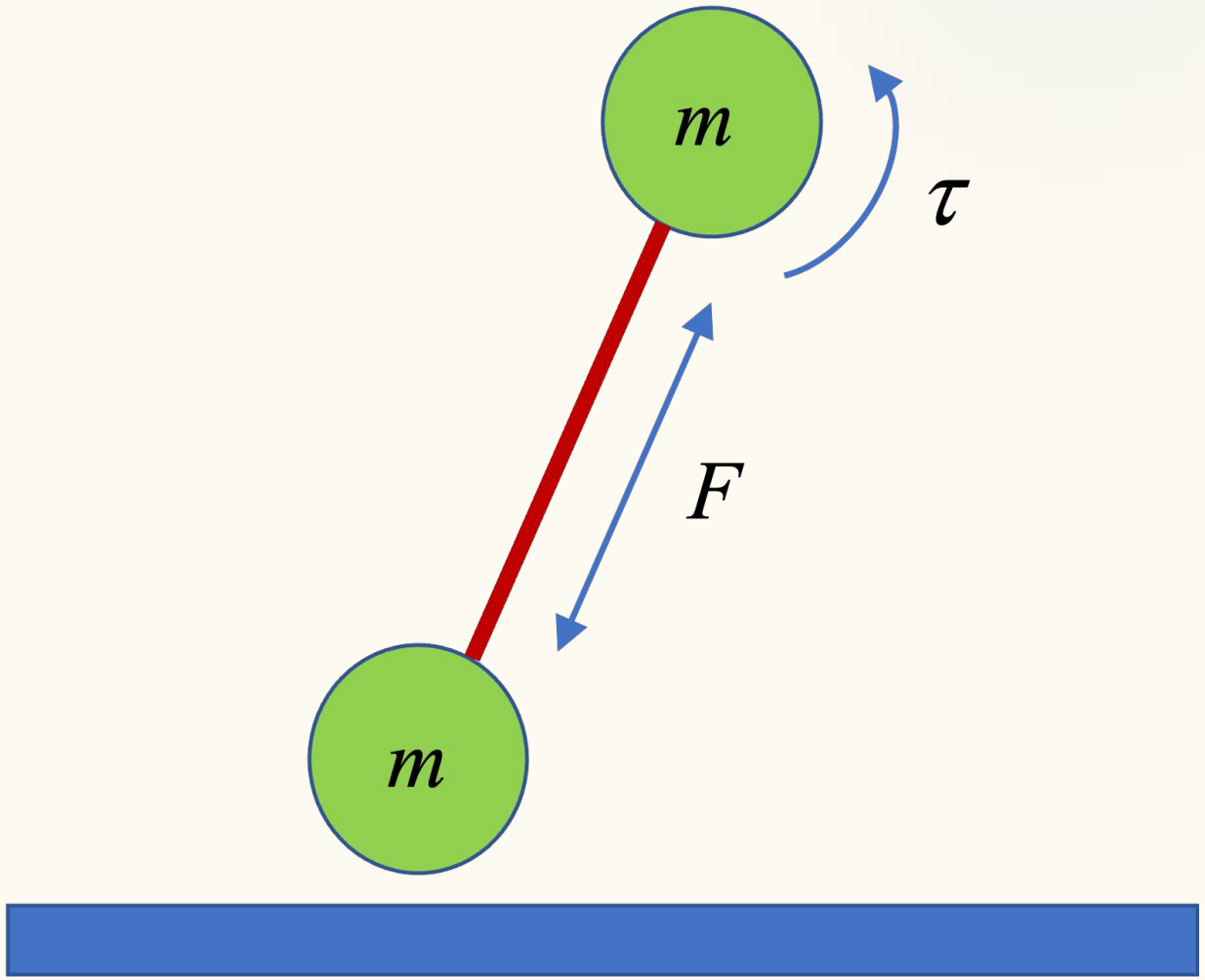
By Tammapon Numsarapatnuck
Kongsakul Tawaitian

Department of Mechanical Engineering

Project Overview

Problem

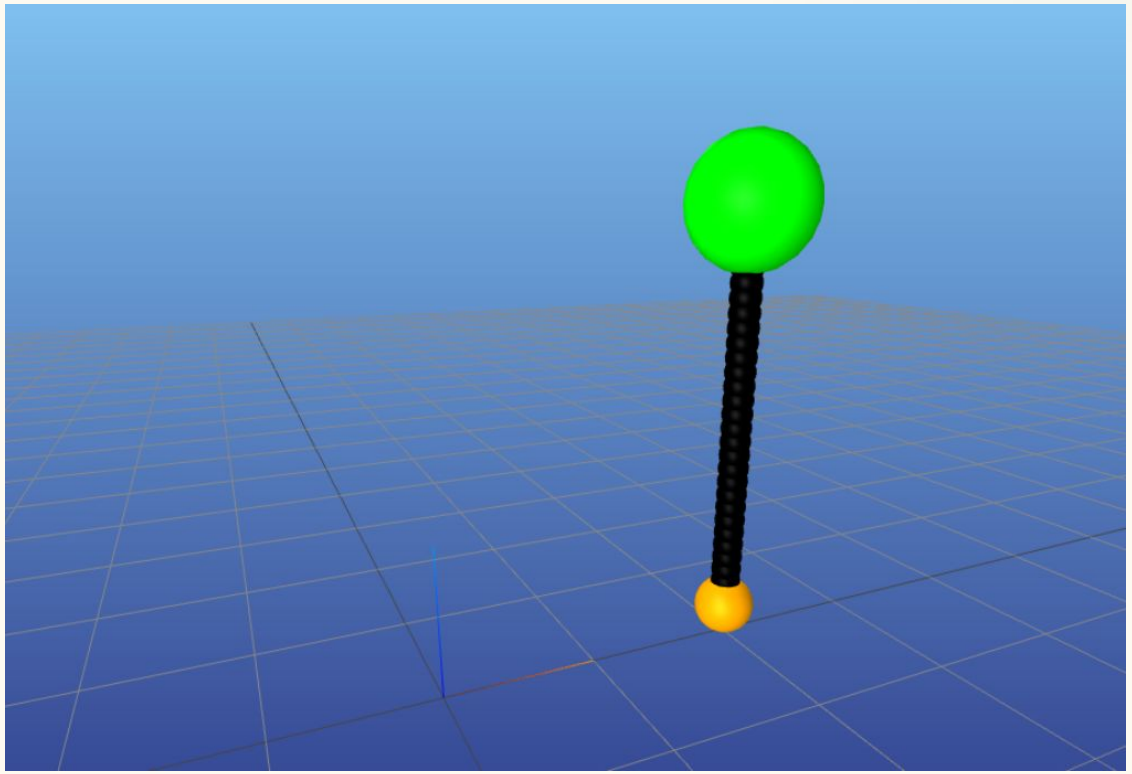
This problem requires finding the hybrid optimal trajectory for a simple robotic leg, referred to as a one-legged hopper, in a 2D plane as shown in Figure P7.1. The structure consists of a robot body mass m and a foot mass that contacts the ground, which are assumed to be equal to simplify the problem. During ground contact, a linear force F along the leg propels the robot into the air, while a torque t is applied to adjust the leg angle for the next touchdown and jump



Defining the Hybrid Dynamics Problem

Robotic Complexity

The One-Legged Hopper represents a classic challenge in robotics due to its **hybrid dynamics**. It operates in two distinct phases: Stance (foot on ground) and Flight (airborne).
This project optimizes trajectories for both 2D and 3D models using Nonlinear Programming (NLP) to achieve stable, energy-efficient jumping.



Core Project Objectives

Hybrid Optimization

To find the optimal force and torque inputs that satisfy jumping goals while adhering to physical constraints like gravity and ground impact.

Key focus: Seamless transitions between stance and flight modes.

Computational Implementation

Leveraging modern Python tools for robust solver performance:

- **JAX:** For automatic differentiation and JIT compilation.
- **CyIPOPT:** Large-scale nonlinear solver interface.
- **MeshCat:** Web-based 3D visual analysis.

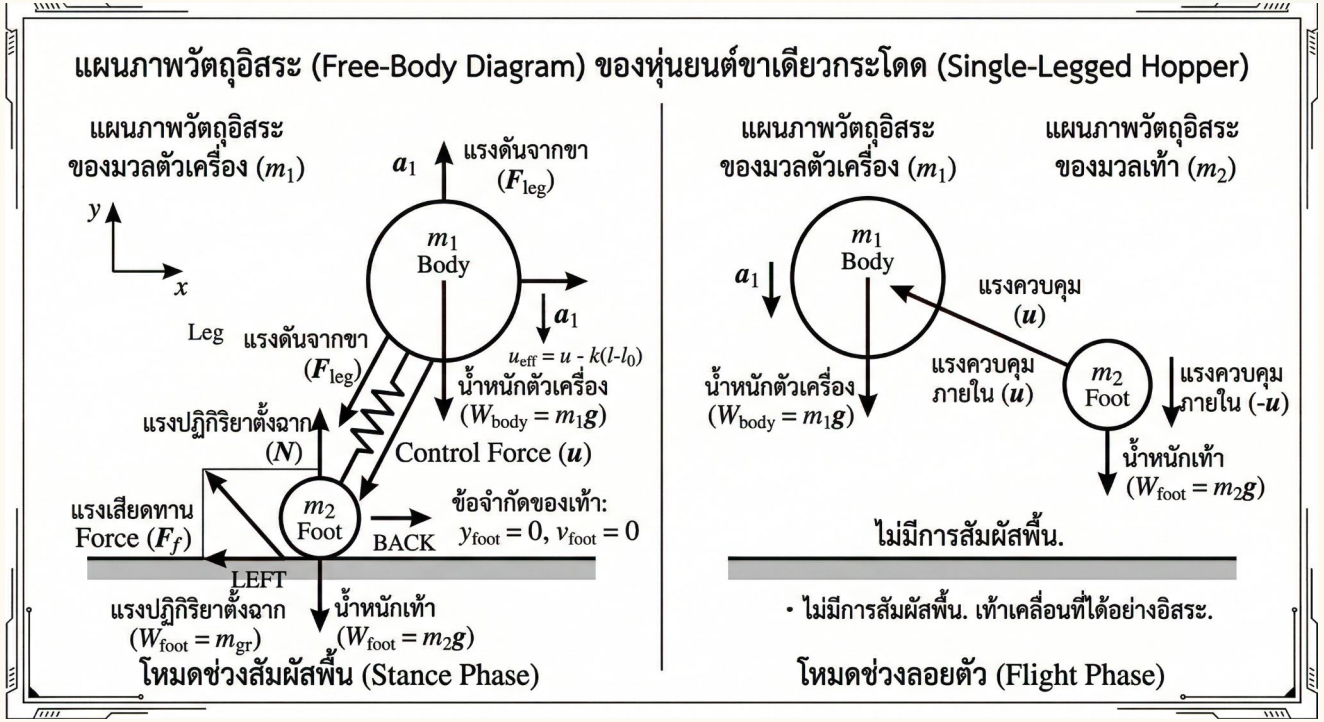
Mathematical System Modeling

State and Control Vectors

The system state vector defines the position and velocity of the robot:

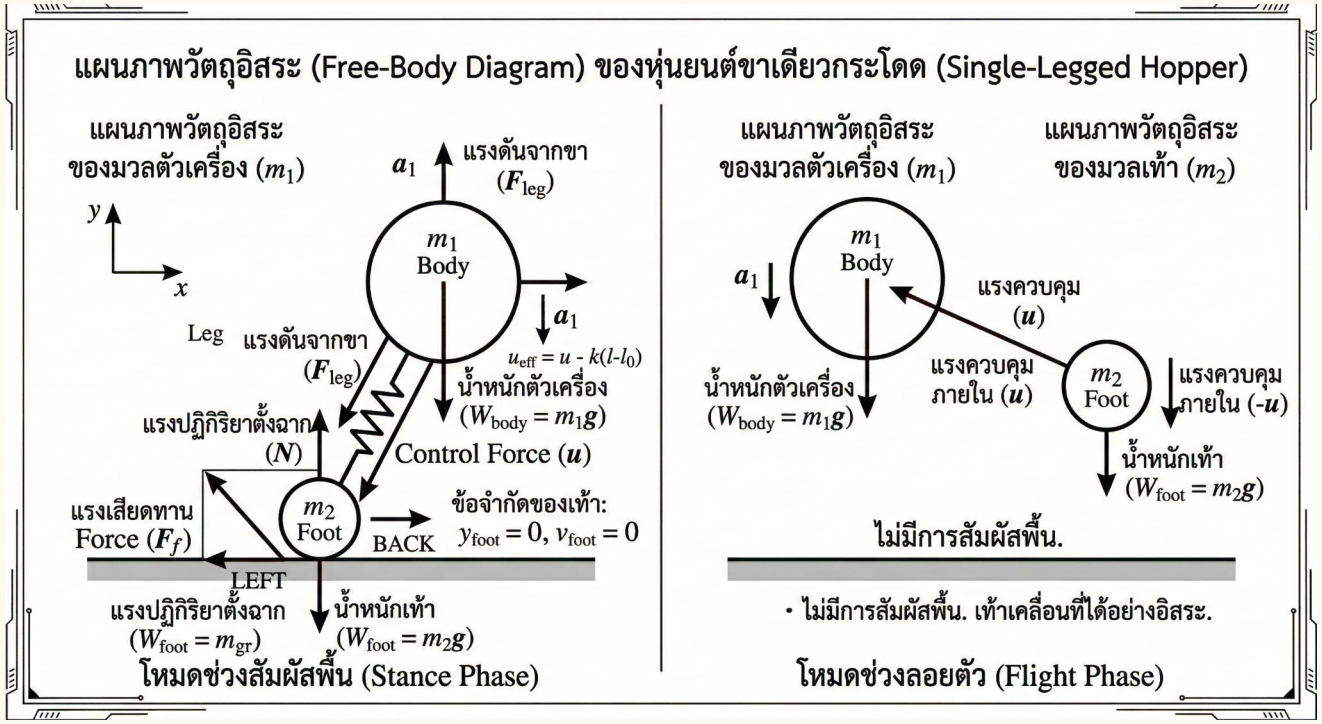
$$x = [\, r_b \,,\, r_f \,,\, v_b \,,\, v_f \,]^T \in \mathbb{R}^8$$

Control inputs comprise radial force and torque .

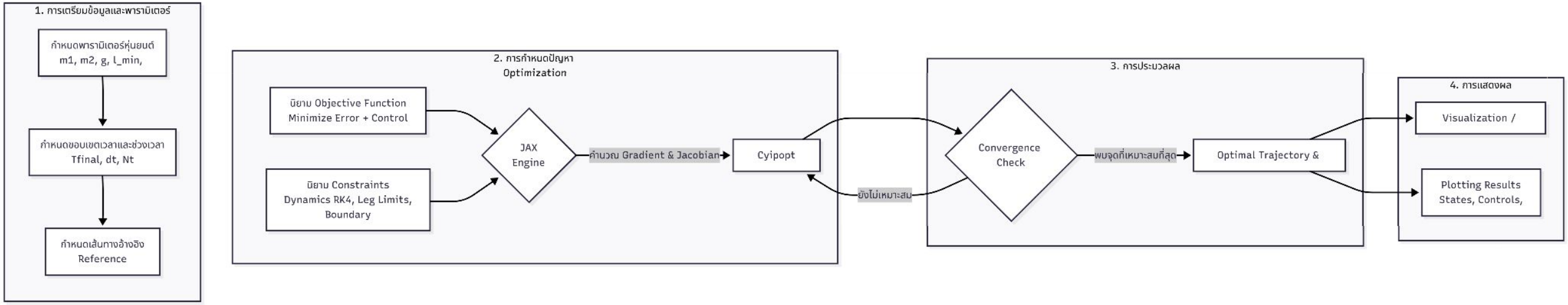


Hybrid Modes and Transitions

- **Stance Phase (Mode 1):** The foot remains fixed on the ground. Vertical constraints ensure . $\varphi(x) = 0$
- **Flight Phase (Mode 2):** The robot is airborne. Dynamics follow projectile motion modified by internal leg forces.
- **Jump Map :** Defines the transition from flight to stance. An inelastic collision model resets the foot velocity to zero.



Workflow


1. การเตรียมข้อมูลและพารามิเตอร์
กำหนดพารามิเตอร์หุ่นยนต์
m1, m2, g, l_min,
กำหนดขอบเขตเวลาและช่วงเวลา
Tfinal, dt, Nt
กำหนดเส้นทางอ้างอิง
Reference
2. การกำหนดปัญหา
Optimization
นิยาม Objective Function
Minimize Error + Control
นิยาม Constraints
Dynamics RK4, Leg Limits,
Boundary
JAX
Engine
คำนวณ Gradient & Jacobian
Cyipopt
ยังไม่เหมาะสม
3. การประมวลผล
Convergence
Check
พบจุดที่เหมาะสมที่สุด
Optimal Trajectory &
4. การแสดงผล
Visualization /
Plotting Results
States, Controls,

Optimization Solver Result

Total Cost

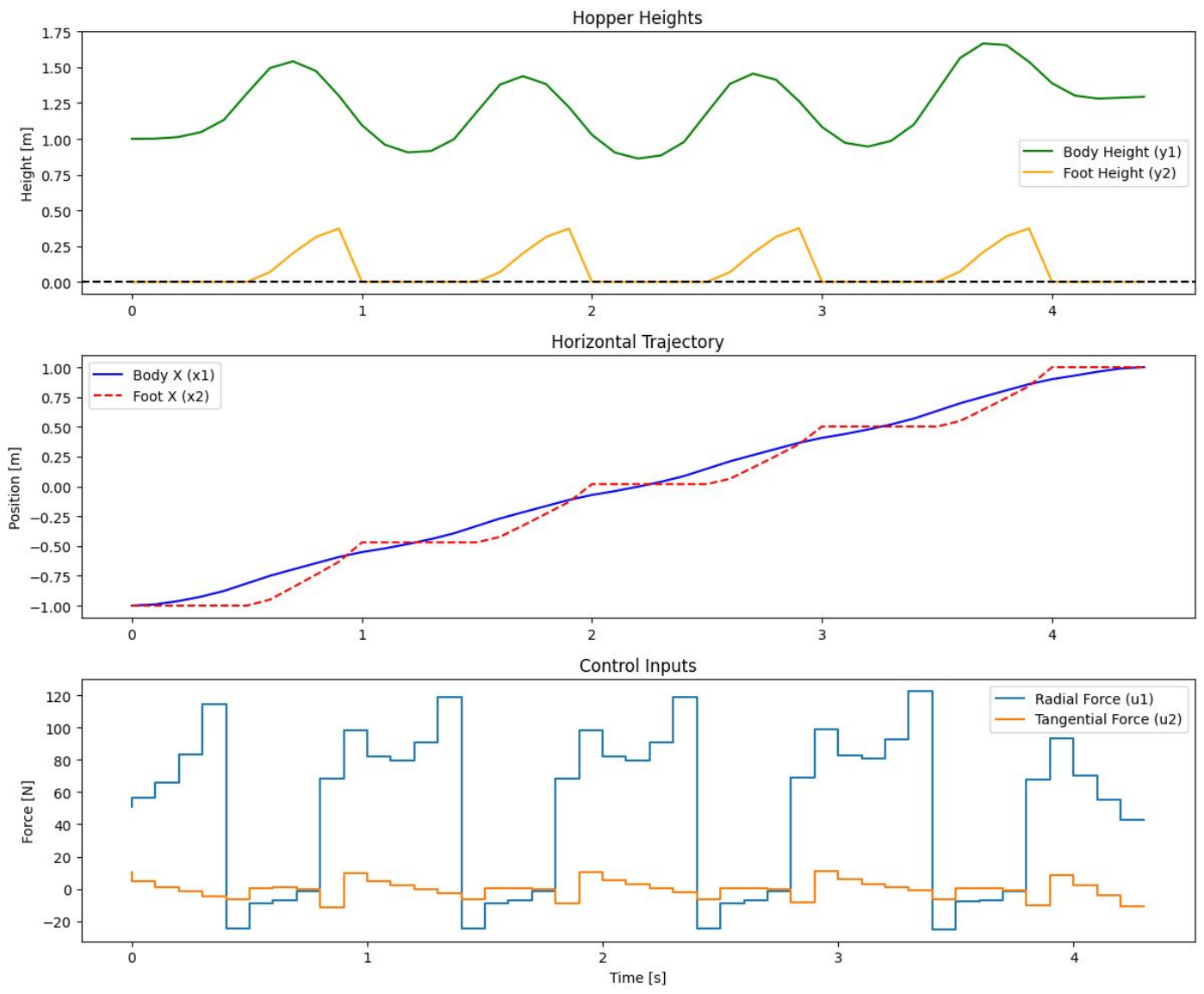
113.928

Optimal Objective Cost (J)

The solver successfully reached an optimal solution in 129 iterations. The resulting trajectory minimizes power consumption and deviation from the reference path.

Newton's equations of motion are satisfied at every discrete time step via RK4 integration.

Converged Trajectory Analysis



Motion Characteristics

The system displays periodic hopping behavior.

Key observations:

- Sinusoidal body height oscillation.
- Steady forward horizontal velocity.
- Precise timing of radial forces during liftoff.

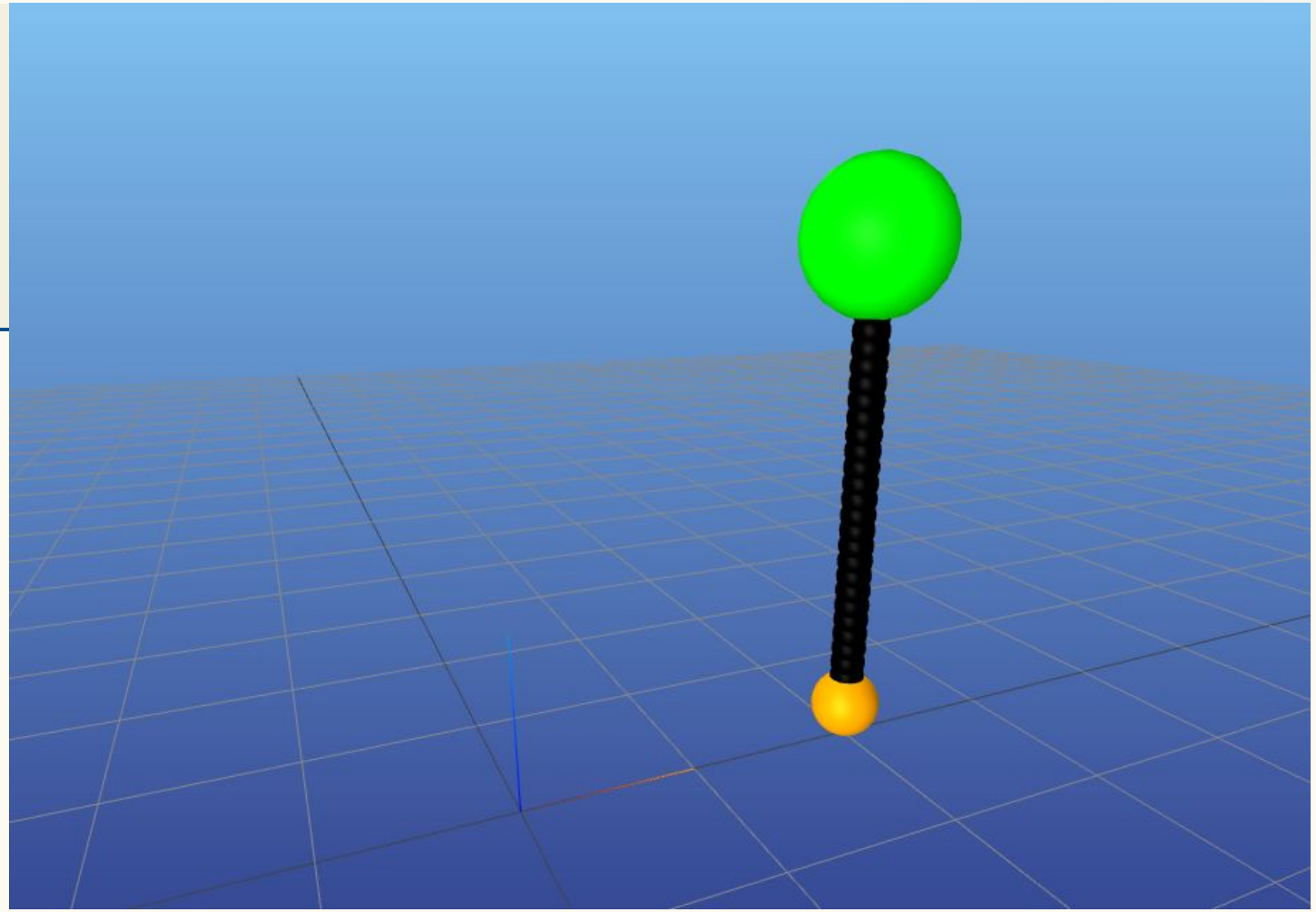
All constraints, including non-negative ground height for the foot during flight, are fully maintained.

Simulation

Visualizing Results

Using **MeshCat**, we rendered the optimized results in an interactive 3D environment.

This allows for direct observation of leg interpolation, impact points, and overall robot stability across the entire 4.4-second simulation window.

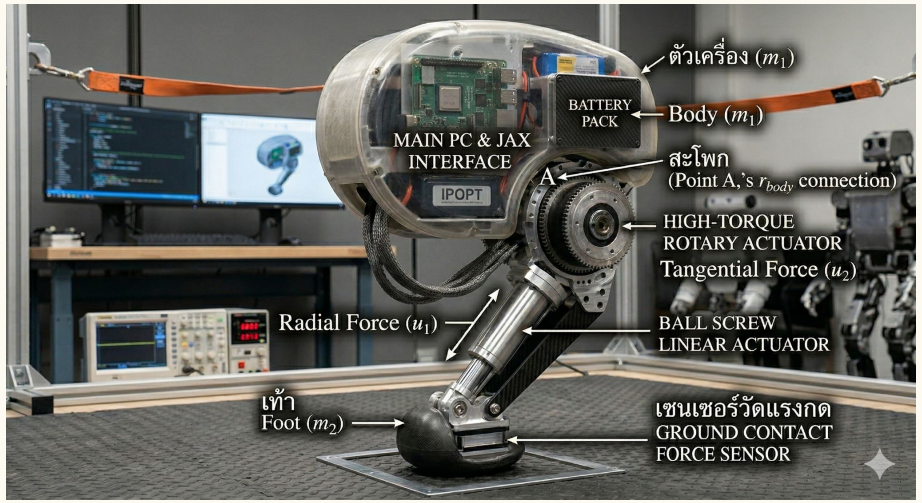


For Implementation

Implementation

Hardware Implementation: From Simulation to Reality

- **Offline to Real-time Control (MPC):**
  - *Current state:* Full trajectory computed once offline.
  - *Hardware solution:* Implement **Model Predictive Control (MPC)**. Run short optimization loops every 10-50 ms (Receding Horizon) to handle real-time disturbances.
- **State Estimator Design:**
  - *Current state:* Assuming perfect knowledge of x, y and v.
  - *Hardware solution:* Use hardware sensors (IMU for acceleration/angles, Encoders for joint positions) combined with an **Extended Kalman Filter (EKF)** to estimate the true state x accurately.
- **Actuator Dynamics Management:**
  - *Current state:* Control input u applies force instantaneously.
  - *Hardware solution:* Apply **Low-level Control** to compensate for real-world motor delays and current limits, ensuring the physical force matches the optimized u command.



Implementation

Challenges	Causes	Solutions
Model Mismatch *(Robot trajectory deviates from simulation)*	The physical parameters (m, l, g) in the code do not perfectly match the actual hardware, or there is unmodeled joint friction.	Perform **System Identification (SysId)** by conducting real-world leg-swinging experiments to collect data and tune the model's parameters to closely reflect reality.
Latency & Computation Time *(Computational bottleneck)*	The optimization solver (e.g., Ipopt) may be too slow to run within a high-frequency real-time control loop (e.g., 100Hz).	1. Implement a **Warm-start** strategy (using the previous optimal solution as the initial guess for the current step). 2. Replace Ipopt with faster, real-time solvers like **OSQP** or **Acados**. 3. Reduce the mathematical complexity of the model.
Impact & Ground Contact *(Excessive collision forces)*	The inelastic collision model used in the simulation might generate abrupt forces that could physically damage the hardware upon landing.	Implement **Impedance Control** (simulating virtual spring-damper dynamics at the leg) during the touchdown phase to act as a landing buffer and absorb the impact force.
Measurement Noise *(Fluctuating state estimation)*	The hardware IMU sensors are inherently sensitive to vibrations and mechanical noise during operation.	Apply a **Low-pass Filter** or fine-tune the noise covariance matrices (Q and R) in the **Kalman Filter** to smooth the signals before feeding them into the controller.

Experimental development

3D Experimental Extension

Extended State

Upping the state to 12 variables (3D position and velocity) and 3 control inputs for true spatial navigation.

Orientation

Integration of rotation matrices and cross-products to handle leg direction in 3D space.

Complexity

Successful convergence in 3D demonstrates the robustness of JAX-based optimization.

Conclusion

Conclusion

Conclusion

This project successfully applied Nonlinear Programming (NLP) techniques alongside Automatic Differentiation (JAX) to compute the optimal trajectory and control forces for a 2D one-legged hopping robot. The model effectively managed the hybrid dynamics transitioning between the stance and flight phases. The results successfully satisfied both the physical constraints and the predefined motion objectives.

The success of the optimization is validated by the minimization of the Total Cost (J), which successfully converged to the optimal solution as shown in the following equation:

$$J_{final} = \sum_{k=0}^{N_t-1} L(x_k, u_k) \approx 113.928$$

Throughout the total duration of T_{final} = 4.4$ seconds, the robot maintained its balance and strictly adhered to the leg length constraint:

$$0.5^2 \leq ||r_{body,k} - r_{foot,k}||^2 \leq 1.5^2, \quad \forall k \in \{0, \ldots, N_t\}$$

Key Findings

- **JAX Efficiency:** Utilizing jax.grad and jax.jacobian eliminated the errors associated with manual derivative calculations and drastically accelerated the solver's processing speed.
- **Control Behavior:** Based on the u_1 (Radial) and u_2 (Tangential) control graphs, it was observed that the system optimally generates maximum thrust just prior to take-off. This behavior effectively converts kinetic energy into momentum for the parabolic flight trajectory.

Limitations and Future Work

Although the model performs exceptionally well in simulation, several aspects require further development before it can be deployed on physical hardware:

- **Model Mismatch:** Real-world environments introduce joint friction and aerodynamic drag, which were not accounted for in the simplified dynamics equation $\ddot{r} = \frac{1}{m}u - g.$
- **Computation Time:** Solving the NLP for the entire trajectory takes too long to be viable for real-time control. Therefore, future development should transition to a Model Predictive Control (MPC) framework, which calculates optimizations over a short receding horizon instead.